X-Ray

X-Ray

A guiding idea for learning, research, and building systems

Flavio · 2026 · Living document · v3b

Prologue: Two Books

In the early seventies I read *Deschooling Society* by Ivan Illich — an Austrian philosopher and social critic who spent his entire life writing about what institutions do to people while pretending to help them.

Illich wasn't writing about bad schools. He was writing about the very structure of institutional learning and why it systematically prevents the thing it claims to produce. Schools — as institutions — don't teach people to think. They teach people to comply. They replace a genuine desire for understanding with certificates, grades, and the illusion of progress.

The book disturbed me. Not because I rejected it — but because I recognised myself in it.

In the early nineties I read *Flatland: A Romance of Many Dimensions* by Edwin A. Abbott, an English schoolteacher who in 1884 wrote a story about a two-dimensional world. The inhabitants of Flatland cannot conceive of a third dimension — not because they lack intelligence, but because their entire perceptual apparatus is built for a reality that simply does not contain it. The prison is not made of bars. The prison is made of reference frames.

That book I recognised too.

These two books shaped not only my worldview but my entire professional and intellectual development. Illich showed me the institutional prison. Abbott the perceptual one. Together they described two dimensions of the same captivity — and I saw both in myself.

I expected that at some point I would arrive at answers. I didn't. I don't fully understand the world around me. I haven't found a complete answer to why and how I do what I do.

I note: not yet. I keep going.

There is a kind of knowledge that is satisfied by describing surfaces. You can know that a flock of birds is flying without ever asking who leads it. You can use an algorithm without ever looking at what is actually happening inside. You can manage a system that works — until it stops working.

That is not enough for me.

X-Ray is the name I gave to my approach to learning and building. It isn't the name of a project or a methodology — it's an attitude. The attitude that it is worth looking inside, even when everything looks fine from the outside. Especially then.

> *I need to see in an x-ray style.* — Joe Strummer, "X-Ray Style" (1999)

Joe Strummer — guitarist, vocalist, and co-founder of The Clash, one of the most influential punk bands of the late seventies and eighties — wasn't singing about medical imaging. He was singing about the refusal to accept what is handed to you as settled fact. The X-Ray style is an act of resistance to opacity — to everything that says: you don't need to know how this works, just use it.

I. Aboriginal X-Ray Art and Knowledge from Within

The Aboriginal peoples of Australia had a particular tradition in visual art now called X-Ray art — stylised depictions of animals in which the internal organs, bones, heart, and lungs are visible. I didn't encounter this tradition in a gallery but while studying epistemology — the philosophical discipline that asks what we can know, how we know it, and where the limits of knowledge lie. One of the standard examples epistemologists use to illustrate that the selection of what we depict is itself a philosophical decision were precisely these images: depict the external appearance of an animal, or depict its internal structure — both are true, but they say entirely different things about what knowledge is.

Western artistic tradition asks: *how does this look?* It tends toward surface realism — light, shadow, perspective — and tries to capture exactly what the human eye sees from a particular vantage point. The Aboriginal painter asks something altogether different: *what does this actually be?* For them, depicting only the skin of an animal means depicting the smallest, least important part of what it is. A true image must show the structure, the flow, the inner logic. That difference is not aesthetic — it is epistemological. This is the X-Ray as an attitude, thousands of years old.

In my projects that question takes concrete form. When I was working on Pong — a browser-based platform for training AI agents to learn to play against themselves — looking at the final result wasn't enough for me. I wanted to know what was happening inside while the agent was learning. So I added an X-Ray overlay: a heatmap showing where the agent was paying attention, real-time telemetry of the ball, a visualisation of the distribution of Q-values — numerical estimates of how good each possible action is at a given moment — as the agent was deciding. The inspiration was Formula 1 broadcast telemetry: the same sport, an entirely different insight.

That is the X-Ray approach in practice. Not just watching the output — but building windows inward.

II. The Black Box and the Grey Gradient

Every time you press a button on your phone and something happens on the other side of the planet — a parcel delivered, a message sent, a transaction processed — there is a space between your finger and that

outcome that is completely dark. I put in money, I get coffee. I type a query, I get an answer. What is in between is someone else's secret or, more often, even the author cannot always explain why the system made that particular decision. Modern technology is an industry of black boxes. And our default relationship with them is surface-level — input, output, don't ask what's in between.

I can't change this. That isn't the goal either. But I long ago stopped assuming that the black box is the only possibility. Between black and white there is a gradient — grey that is transparent enough to let you see something.

My approach is building layers of understanding around an opaque core. It works through two lenses:

Metadata as a mirror: what the system did, when, with what outcome, how it behaved under stress. Metadata — data about data — is not the interior of the black box, but it is its shadow. And a shadow sometimes tells you more than you'd expect.

Meta-learning as a compass: not just what the system does, but what I learn from observing how the system does it. Every observation is both technical and reflective — what happened, and what that says about my understanding.

I have a project called Katalog — a central meta-registry of all my projects, chats, and documents, stored in a PostgreSQL database with tools for direct interaction. It isn't an archive. It's an implementation of this philosophy: a system that illuminates other systems from within. Katalog doesn't ask what I've built — it asks how what I've built changes over time, where the gaps are, what's missing.

The shadow is sometimes enough. But only if you know you're looking at a shadow, not the original.

This is a limit I don't hide. Abbott already warned me about it in the prologue — just in different language. The inhabitant of Flatland observing the shadow of a three-dimensional sphere on their two-dimensional surface sees a circle that appears, expands, contracts, and vanishes. From that shadow they can infer many things: that the object exists, that it moves, that its size changes. But they cannot infer its shape. They cannot infer its depth. They cannot infer the cause of its movement. Metadata is that shadow. It tells me what the system did, when, with what outcome — but the causal core remains out of reach. Documenting a failure and tracking a recovery is not the same as understanding why the failure occurred. X-Ray philosophy doesn't claim otherwise. It promises a better view than you would have without it — not omniscience. Knowing what you don't see, and why that isn't a defeat — that is the X-Ray attitude toward limits.

III. Meta-Learning: How to Learn What You're Learning

Every time I start working on something new, I notice that I'm drawn to the same goal that precedes the technical details: understanding how to understand what I'm facing. That is meta-learning — learning about learning. If epistemology, which I wrote about in the previous chapter, is the question of what we can know and where the limits of knowledge lie, meta-learning is the question one step closer to the process: how do we build that knowledge, by what path, and how can we improve that path. The difference matters — epistemology describes the map, meta-learning teaches you to walk.

Meta-learning isn’t abstract philosophy. It’s a practical set of questions you ask at the very start: Which concepts exist here? In what order does it make sense to understand them? Which analogous systems can help me orient myself? What will I know when I truly understand this thing — how will I recognise it?

Meta-learning brings with it the need for metadata. When you’re training an AI agent, it isn’t enough to watch the final result. How many episodes did it take to learn something? In which situations does the agent still make mistakes? How does the behaviour change over time? These secondary layers of information are windows into the interior. They aren’t the heart, but they’re closer to it than the result alone.

Every project I start has the same hidden goal: to understand not just what I built, but how I understood what I built.

This isn’t a meta-comment that comes at the end of the project. It comes at the start. When I began working on the SIR project — Self Improvement & Research, research into agentic AI, fine-tuning, and PostgreSQL infrastructure — the first thing I set up wasn’t the database. It was the question: what is the smallest step of understanding I need in order to know what to ask next? That habit changes the project itself. You move differently when you know that understanding the process matters as much as the result.

IV. Emergence: Order That Nobody Programmed

A flock of a thousand starlings changes direction in a fraction of a second, perfectly coordinated, without a single leading bird. Each starling follows only three basic rules with respect to its neighbours: not too close, not too far, same direction. Nobody wrote an algorithm for the flock. The flock simply happened.

This is emergence — global order arising from local, simple rules of interaction. No architect. No centre. No plan. An ant colony builds complex structures, solves optimisation problems, organises its food supply — from the same principle. Complexity does not require complex instructions.

Emergence fascinates me for two reasons that seem contradictory at first glance. First: powerful systems sometimes arise from minimal principles. You only need to define the rules of interaction and let the system organise — there’s no need to program every detail. Second: a system can exhibit behaviour we didn’t anticipate or design, and that behaviour can be both brilliant and pathological. Emergence is both promise and threat — and both at the same time.

I saw this directly in Pong. During the self-play training phase — where the agent plays against itself, without any human opponent — it began developing strategies I hadn’t programmed. It didn’t just learn to return the ball. It learned to set traps, deliberately drive the opponent into corners, use the edges of the playing field. These behaviours weren’t in the reward function — the numerical signal that tells the agent whether it did something good — they emerged from the optimisation process itself. I watched something arise from rules that wasn’t in the rules.

The X-Ray view of an emergent system doesn’t try to track every element — that’s impossible and actually the wrong question. It tries to identify the fundamental rules from which complexity arises.

Understanding the rules is more important than describing the results. Results change. Rules explain why.

What emergence is not — and it's worth saying because it is often confused — is self-organisation. Emergence describes what arises: global order that wasn't explicitly programmed. Self-organisation describes the forces that organise it: the dynamics of a system that finds stable forms without external control. That distinction isn't terminological pedantry — it opens entirely different questions, and that is what the next chapter is about.

V. Self-Organisation: The Forces That Build and the Forces That Trap

Related to emergence, but different. Emergence tells me what arises — the flock, the anthill, the agent's strategy. Self-organisation tells me why precisely that arises, and not something else: what forces are at play, what the dynamics of the system are, and — crucially — where those forces can become a trap.

Crystallisation, galaxy formation, the development of living organisms, social norms — these are all examples of self-organisation. A system without external control finds stable forms from its own dynamics. Order grows from within. That is both fascinating and, when I look more carefully, a little unsettling.

The unsettlement comes from the dark side of self-organisation, which is as real as the beautiful side: systems self-organise toward a local optimum, not necessarily a global one. An ant will follow a pheromone trail that leads in a circle — the same force that builds the anthill can lead the colony to its death. This isn't a bug in self-organisation. It is its nature. Powerful and unreliable simultaneously, and impossible to separate.

I saw this dark side in Pong during the DQN phase — where I introduced a deep neural network instead of tabular Q-learning. DQN stands for Deep Q-Network, an architecture that allows an agent to learn from raw pixels instead of manually defined states. The agent began finding solutions that optimised the reward — the numerical praise signal — but were useless as a game strategy. It learned to stand still because movement never brought a fast enough reward. It self-organised into a local optimum from which it couldn't escape. I call this phenomenon a death spiral — a system that is technically stable but functionally dead.

The death spiral wasn't a failure. It was the perfect outcome of wrong rules. The system did exactly what it was optimised to do — and that is precisely why it didn't do what I wanted.

This isn't just a problem for AI systems. A student so afraid of a bad grade that they stop coming to class has locally optimised: no class, no bad grade. They have globally failed: no knowledge, no progress. An employee who never proposes anything new because every proposal might be rejected — stable, safe, functionally dead. The same mathematical mechanism. The same trap. The X-Ray attitude toward self-organisation means asking: is this stable pattern a true optimum, or only a local one?

The X-Ray approach to self-organisation means trying to understand the forces that organise a system — not just describing what arose, but why precisely that arose. When I see a stable pattern in a system

I’m learning, I don’t celebrate it immediately. I ask: is this a true optimum or a local one? What would need to be different for me to tell the difference? What rules produced this form?

Self-organisation taught me that stability is not proof of correctness.

VI. Genetic Algorithms: Evolution as Method — and as Warning

Darwin’s evolution is perhaps the most beautiful algorithm that exists. Variation, selection, inheritance — three principles that, through billions of iterations, pulled an elephant, an orchid, and a nervous system out of a chemical soup. No designer, no plan, no goal except local survival.

Genetic algorithms are an attempt to apply that logic to solving computational problems. Instead of seeking the optimal solution directly, you generate a population of possible solutions, evaluate them, combine the better ones, eliminate the worse ones, introduce a little randomness, and repeat. The results are sometimes surprising — the algorithm finds solutions a human would never have thought to look for, along paths entirely different from those that would seem intuitive to us.

In Pong v3.0 I implemented exactly this: AUTO-GENERATE, which creates a population of agents with different hyperparameters — the numbers and values that control how an agent learns — TOURNAMENT, which pits them against each other, and EVOLVE, which combines the genes of the winners and introduces mutations for the next generation. I didn’t program a strategy. I programmed the conditions under which a strategy could evolve.

What interests me about genetic algorithms isn’t just their practical application but what they say about the nature of optimisation in general. A local optimum is a trap — a solution that is better than everything in its vicinity but isn’t globally best. Mutation in genetic algorithms serves precisely this: to escape the local optimum and explore unknown parts of the space of possibilities. Without mutation, the population converges. Convergence looks like success. Sometimes it is. Sometimes it’s just a more elegant form of stagnation.

Then I noticed something that stopped me.

I too am in danger of a local optimum in learning.

When a technique or approach starts working, the tendency is to stay with it. I know it. It works. It delivers results. Why change? But that very logic — which is perfectly reasonable in the short term — is what prevents me from ever finding out whether there’s a better solution on the other side of the valley. Genetic algorithms have become a reminder that periodic exploration of the unknown — even when it looks like a waste of time, even when it doesn’t immediately deliver results — is what prevents intellectual stagnation.

Here I need to be precise: mutation as a principle holds in the space of exploration and learning. It doesn’t hold everywhere. A pacemaker that randomly mutates its rhythm to check whether a new one is better should not exist. A nuclear plant that experiments with a new safety protocol in production isn’t an X-Ray attitude — it’s irresponsibility. The value of mutation depends on the cost of error in a given system. Where the cost of error is low, exploration is

imperative. Where it is high, mutation must be isolated, controlled, tested. And that limit isn't a weakness in the argument — it is part of it.

Mutation is not an error in the process. Mutation is the process.

VII. Resilience: Failure as a Design Requirement

There is an illusion, especially prevalent in engineering disciplines, that the goal of a system is to eliminate errors. Sufficient redundancy, enough tests, enough validation — and the system will never fail.

That is an illusion.

High-reliability systems — HA systems, as we call them in engineering jargon, from High Availability — are not reliable because they don't fail. They are reliable because they are designed to survive failures and recover from them autonomously, without human intervention. A Linux cluster that detects a node failure and automatically switches the service to another node, a database that replicates and continues working while the primary node recovers — that is HA philosophy in practice: not preventing failure, but ensuring the system continues to function while the failure is being resolved.

From my most concrete experience with failure — the Balsam server crash due to the Linux OOM Killer, 9.5 hours of unavailability at three in the morning — I drew one insight that I now carry as a design principle: *the only component that survived was the rsync backup that ran autonomously and was architecturally isolated from everything that could fail.* It wasn't smarter. It didn't have more resources. It was isolated. Isolation was the only difference between what survived and what didn't.

That insight changed how I design every new system: I don't ask myself 'what if this doesn't work?' as an edge case. I ask it as a central design requirement. What is the minimal functional set? What can fail while the system keeps running? How does the system behave under degraded conditions?

This isn't pessimism. This is precision. A system designed under the assumption of perfection becomes fragile. A system designed under the assumption of failure becomes robust.

Technical details of the server crash and examples from the Katalog project are in the Appendix.

Running on the Balsam server is ntfy — a push notification service that sends alerts to a phone via simple HTTP requests. What from the outside looks like a single service is, internally, dependent on a PostgreSQL database. When the database stops working for any reason, ntfy stops working. Silently, with no error message going out.

But what if the entire Balsam server goes down? Then there's neither primary nor fallback from Balsam. In that case the Termux devices step in — Android phones and tablets running independently of Balsam. They don't report that they're alive. They report only one thing: that Balsam isn't. That message reaches the public ntfy.sh, the only channel that is reliably working at that moment.

Three layers, without anyone's intervention. A system that doesn't wait to be noticed.

Fast recovery is better than perfect prevention — this is not a compromise. It is better engineering.

VIII. Neuroplasticity: The Brain That Mutates

I wrote about genetic algorithms as a method that uses forced mutations to escape a local optimum. I wrote about resilience as a system that assumes failure instead of fighting it. But I wrote all of that about code, about servers, about agents learning to play Pong.

There is one system that does all of this — and that I've been running much longer than I've been writing code. My own brain.

A neuron on its own does almost nothing. With an impulse it activates neighbouring neurons, releases neurotransmitters, receives a signal or doesn't. That is the whole story of one neuron. But when you connect ninety billion of them in a network, something happens that isn't in any individual neuron: understanding emerges, language, emotion, strategy. From minimal local rules arises a complexity that writes symphonies, climbs Everest, and builds LLM models. The same emergence as in the flock of birds — but the substrate is flesh, not code.

What makes the brain special isn't just what it emerges. It's that it reorganises.

Neuroplasticity is the brain's ability to physically change its structure in response to experience. Synaptic connections that are activated grow stronger. Those that aren't used — atrophy. The brain isn't a fixed architecture; it is an architecture written by experience. This is the biological equivalent of what I saw in Pong v3.0: a population of agents evolving through selection. In the brain, selection isn't programmed — it happens by itself, every time you learn something, repeat it, or stop doing it. Every new skill builds new synaptic pathways. Every abandoned skill allows those pathways to weaken.

The problem arises when synaptic pathways become so worn in that you start mistaking them for truth.

For years I worked on a database system that had no technical ability to rename objects. No rename — only copying to a new object with a new name, then deleting the old one. When you have a database with several hundred million records, that procedure becomes a nightmare: double the space, double the time, double the risk.

I knew that system well. I had attended courses, worked on projects, knew every limiting factor. And I was working, by every measure, as well as possible — within the given rules.

A younger colleague joined the project. His courses were planned for the following quarter, he had no experience. We sat and discussed the problem. In the end, almost naively, he asked the question: why don't we just change the name directly in the data dictionary?

Silence.

In every manual, at every lecture, it said the same thing: the data dictionary is taboo. We don't touch it. We don't know what will happen. There was even a threat of losing a high-value licence.

But the question had been asked. And I — for a few seconds that felt like an eternity — performed an X-Ray on that taboo. It wasn't a technical impossibility. It was an institutional reflex instilled through repetition, worn into synaptic pathways to the point where no one was checking the premise anymore.

I said yes. The job was done in a few seconds, with no downtime. Nothing bad happened.

That night, before a full backup, I went through the entire data dictionary and its relationships — an X-Ray session of a system whose taboo we hadn't examined for years. And I showed my younger colleague how I do it.

Schopenhauer, when criticised for preaching humility while living lavishly, replied something like: does a sculptor have to be as beautiful as the statues he carves? I apply the same cynical logic to myself: the person who writes about the X-Ray attitude need not be immune to institutional reflexes instilled by years of experience. Expertise and blindness aren't opposites — sometimes they are the same synaptic pathways.

The younger colleague didn't have my knowledge. But he didn't have my worn-in inhibitions either. His naivety was, at that moment, an advantage. He was a child in Nietzsche's sense — he hadn't yet taken on the camel's burden.

In *Thus Spoke Zarathustra* Nietzsche describes three metamorphoses of the spirit: the camel, the lion, the child. The camel carries the burden — conventions, obligations, rules. The lion fights against the burden and refuses it. The child is the freedom on the far side of the struggle — the one who can create new rules, not merely refuse old ones. Why the child, and not the adult who has won? Because the child is not yet captive. The prefrontal cortex — the part of the brain that matures last, somewhere in the late twenties, with some newer research pushing that boundary toward thirty-two — is the seat of social norms, long-term planning, and impulse inhibition. It is the neurobiological substrate of the institutional prison Illich writes about. School, culture, professional socialisation — all of it writes rules into the prefrontal cortex while it is still plastic. By the time it has matured, many rules have become part of the black box — you don't see them, because they have become the reference frame.

The child doesn't yet have that filter. Not because they're smarter, but because the filter hasn't been installed yet. That is both an advantage and a vulnerability — but at the right moment, that openness sees what the expert cannot. The X-Ray attitude isn't a return to childhood. It is the capacity of an adult to, at a chosen moment, suspend the filter and look as though it isn't there.

If a genetic algorithm needs mutation to escape a local optimum, the brain needs the same. And, unlike code, you can initiate it consciously. Change your route to work. Not every day — occasionally. It doesn't matter whether it's shorter or longer. What matters is that the brain can't use the automated, worn-in path. The new route activates the prefrontal cortex and hippocampus — the brain is forced to be present. Hide the mouse. Learn to use the keyboard in ways you avoid. Or the reverse — hide the keyboard and use the mouse for things you normally do with keys. Learn to knit, or to play something on the piano, or learn a dead language. Latin and Greek, which are now learned without the pressure of communication, activate analytical capacities in a way that living languages cannot — there's no native speaker judging you, no culture pressing in on you. All of

this is artificial mutation initialisation. The goal isn't to become an expert knitter. The goal is to prevent the brain from completely freezing into its current architecture.

Genetic determinism is a myth that needs to be seen through with X-Ray. Genes are a predisposition — a recipe. But the recipe doesn't determine the dish. Environment, experience, habits, decisions — that is the cook who interprets the recipe. The same genetic material, different environments, different synaptic architectures, different lives. Neuroplasticity is the mechanism through which the cook works. Every decision to learn something, every deliberate mutation of routine, every moment when we ask why this is taboo — all of it changes the physical structure of the brain. Not metaphorically. Literally.

Understanding this is not a reason for optimism or pessimism. It is a reason for attention. The brain you put through X-Ray — your own black box — isn't static. It changes with what you do with it. And with what you don't.

Sometimes the most valuable X-Ray you can perform isn't in code, isn't on a server, isn't in a data dictionary. It is inside.

And that question — what do I see when I turn the X-Ray on myself, on my own cognitive biases, decision habits, and blind spots — isn't a question for the end of a project. It comes at the start. And it changes how I walk through everything else.

X. Wittgenstein and Transformers

Somewhere around 1919, Ludwig Wittgenstein was convinced he had finished the job. *Tractatus Logico-Philosophicus* — a book written in the trenches of the First World War — claimed that all philosophical problems are, at their core, problems of language. And that now, with that clear, they were solved.

He didn't just write this in a book. He lived that conviction. He renounced his family's fortune. He abandoned philosophy. He became a village schoolteacher, a gardener, an architect. For almost an entire decade.

This wasn't humility. It was a logical consequence: the work is done, now what?

He returned in 1929, after presenting to the Vienna Circle — and in the years that followed he wrote *Philosophical Investigations*, a fundamental correction of everything he had claimed in the Tractatus. Not an extension. A correction. He said: I was in a trap. Language is not a mirror of reality — language is a game. Meaning is not in the form of a sentence but in its use, in context, in the life that surrounds the sentence.

The philosophical community was astonished. Wittgenstein was not.

This is the X-Ray attitude in its purest form: not a declaration that you were wrong — but an act. He left philosophy as a man who had finished the work. He returned as a man who had seen from the inside that the work was not what he had thought. No dramatic announcement is needed. A look inward is enough.

The ideas from *Investigations* — meaning as use, context as the basis of understanding, family resemblance between concepts — became the foundation of modern linguistics and, indirectly, natural language

processing. When a transformer computes attention between tokens, it is implicitly doing what Wittgenstein described in words: looking for contextual connections, not abstract definitions. Wittgenstein did not anticipate transformers. But transformers are, architecturally, Wittgensteinian. The idea survived self-correction and reappeared in a form its author could not have imagined.

How does a man who thought he had closed all questions become the cause of opening entirely new ones? Because he had the courage to look inward — and change direction.

XI. Attention Is All You Need

In 2017, eight Google researchers published a paper whose title was an arrogance that proved justified: *Attention Is All You Need*. The transformer architecture — an attention mechanism that computes relationships between all tokens simultaneously, in parallel, without recurrence — was a leap forward. By 2025, that paper had been cited more than 173,000 times, placing it among the ten most-cited papers of the 21st century.

It was an elegant solution. And like every elegant solution, it carried within it the seed of its own limits.

Transformers require computing time that is quadratic in the size of the context window. Every token must be compared with every other token. Double the context — quadruple the computation. This is not an implementation flaw. It is the mathematical nature of the attention mechanism. The wall has been known from the start. And from the start the direction it leads has been known: larger models require more data centres, more electricity, more cooling systems. The industry responded with scaling — more of the same, not something different. Alternatives exist — hybrid architectures, state space models, linear attention — but every approach that operates in subquadratic time fundamentally loses some capabilities the transformer has. Quadratic complexity is not a bug waiting for a patch. For some tasks, it is the price of understanding itself.

And yet — the industry keeps building power plants. Because that is the path we know.

There is a habit that has helped me not lose my bearings in this field: when an AI problem captures my attention, I look for sources from before 2017. I look for how people thought before the dominant paradigm. Not because old is better — but because every paradigm has blind spots toward which the one that came before it was not blind. The X-Ray view of the present sometimes requires a passage through the past.

At some meeting, somewhere, a younger colleague who hasn’t yet learned how “things are done” could ask: why don’t we change the very definition of what we’re asking the model to do? Instead of scaling a quadratic problem, perhaps the question isn’t wrongly solved — perhaps it’s wrongly posed. Wittgenstein was that child for himself. He had only paper and the will to demolish his own system. The AI industry builds power plants to do more of the same.

That child hasn’t come yet. Or it has come and there was no one to hear it.

When AI develops exclusively through scaling, the only ones who can participate are those with resources. And resources are not evenly distributed. Languages spoken by more than 50 million people are classified as “low resource languages” — not because few people speak them, but because they are absent from the digital space. Fine-tuning requires expensive hardware. Software written for it won’t even launch without a GPU. Knowledge embedded in languages the model has never seen, in cultures not represented in the training set — for AI, that knowledge does not exist.

The entire AI field is asymmetric toward knowledge drawn from sources of the global west and, practically speaking, two dominant languages. This is not a coincidence — it is the consequence of architectural single-mindedness that becomes embedded in the values and assumptions the model learns alongside its data.

SIR — my project for researching agentic AI and fine-tuning — emerged in part from this frustration. An attempt to build, without a data centre and without a GPU, with an X-Ray attitude toward available resources, something that would otherwise require the infrastructure of the global north. This is not a political statement. It is the same attitude as in the data dictionary: why not look inside and check whether the taboo is a technical impossibility — or merely an institutional reflex?

IX. Synthesis: X-Ray as a Value System

All of this — Aboriginal X-Ray art, Strummer’s rebellion, meta-learning, the black box and the grey gradient, emergence, self-organisation, evolution, resilience, neuroplasticity, Wittgenstein’s metamorphosis, transformer single-mindedness — is not a random collection of interesting ideas. These are facets of the same attitude toward knowledge and toward building.

Illich showed me that institutions can systematically prevent the very thing they claim to produce. Abbott showed me that a prison can be made of reference frames, not bars — and that there is no exit from it until you assume that there is a dimension you don’t see. I didn’t read those two books as academic literature. I read them as diagnoses.

The X-Ray approach doesn’t promise complete understanding. It promises a better view than you would have without it. Layer by layer, metadata by metadata, observation by observation — I build an understanding of systems that are inherently opaque. And I know that understanding is always partial. The shadow of the sphere in Flatland isn’t the sphere — but it tells me the sphere exists, that it moves, that there is some motion I can track. That isn’t nothing. That is sometimes everything we have.

Emergence and self-organisation remind me that I don’t need to control every detail — what matters is understanding the rules from which behaviour arises. But they also remind me that stability is not proof of correctness, and that the same forces that build an anthill can lead a colony in circles.

Genetic algorithms remind me that stagnation is more dangerous than wandering. That convergence looks like success and sometimes is — but that without mutation I can never know whether there’s a better solution on the other side of the valley where I’ve comfortably settled.

Resilience reminds me that perfection is neither achievable nor necessary. That the only component that survived my server crash was the one architecturally isolated from everything that could fail. And that that lesson cost 9.5 hours of unavailability at three in the morning.

Neuroplasticity reminds me that the hardest black box is the one I carry with me. That expertise and blindness aren't opposites — sometimes they are the same synaptic pathways. And that the X-Ray attitude is, in the end, the ability to turn the apparatus on yourself — and look.

Wittgenstein reminds me that not even the most closed system is final — if you have the courage to look inward. Transformer reminds me that the elegance of a solution is no protection against the single-mindedness that follows from it.

Meta-learning is the thread that runs through everything: I always ask not just what I'm learning, but how I'm learning it, and how I could learn it better. That question isn't a meta-comment that comes at the end of a project. It comes at the start. And it changes how I walk through everything else.

. . .

Look at the systems you use every day. The tools that have delivered results for so long that you've stopped checking the premises. The habits that have stabilised to the point where you no longer see them as choices. The expertise that has become a reference frame.

Are you in a local optimum? How would you know?

Stability is not proof of correctness. A system that doesn't fail may be in perfect stagnation. An expert who never makes mistakes may be a person who has stopped exploring.

X-Ray isn't an answer. It is the attitude with which we approach questions.

. . .

Every project is an experiment. Every experiment is a data point. Every data point is one pixel in the X-Ray image of what I'm building.

This document is alive.

It isn't finished at the moment of writing — it will be finished with new experiences, new projects, new insight. Places for future expansion:

→ *Briscola and DQN — partial observability as an X-Ray problem: an agent learning to infer what it doesn't see, building a model from available traces.*

→ *Katalog as a living archive — a meta-registry that illuminates all other projects from within, an implementation of X-Ray philosophy in a database.*

→ *Failure modes as philosophy — documenting not just what worked, but what failed and how the system recovered. That is X-Ray documentation.*

→ *Quantum computing and cellular automata — new terrain for the same questions: what is inside, what are the minimal rules, how does complexity arise.*

Appendix

A Note on the Laboratory

All of the examples that follow emerged from my personal sandbox — an isolated environment in which I develop projects, test ideas, and learn from failures. The consequences of errors and crashes remain within that environment and do not affect the outside world, except sometimes my sleep when they happen at three in the morning.

Project James: The Matchstick in the Doorframe

In James Bond films there is a scene that repeats: before leaving his hotel room, Bond discreetly wedges a matchstick into the doorframe. When he returns, he checks whether the matchstick is still there. If it isn't — an enemy was there, or is still inside.

Same principle, different substrate.

James is an application on a mobile device. The device can be attached to a door handle, placed in a nightstand drawer, leaned against a window, or hidden in a briefcase with important documents. It monitors linear acceleration — a signal from which gravity has already been removed, so it responds to touch rather than the building's vibrations. When someone or something moves the object, an ntfy notification reaches the owner's phone within seconds.

No camera. No cloud services. No special hardware. Everything that already existed in the ecosystem — redeployed into a new function.

The X-Ray view of James is not technical — it's an attitude. Bond's matchstick in the doorframe wasn't a security system. It was a state indicator. James does the same: it doesn't protect, it illuminates. It tells you what happened while you were away.

Balsam Server: When the Kernel Kills a Process

During the night of 23–24 March 2026, my Balsam server froze and stopped responding. The SSH connection dropped. The VNC console was inactive. The server was unavailable for 9.5 hours, from 03:00 to 12:30.

The cause: the Linux OOM Killer — a kernel mechanism that, when the system runs out of free memory, finds and kills the process consuming the most RAM — killed the Ollama Docker container, which was occupying around 30% of the total available RAM. On a system without a swap partition the only exit was SIGKILL. The server froze immediately afterward.

The prior assumption was simple: three services can coexist on the available RAM without explicit limits. Nobody had accounted for the possibility that at peak consumption they might simultaneously request more than was available, and that without swap the kernel has no alternative but to kill.

The failure wasn't detected in real time. I discovered it in the morning when SSH wouldn't connect. Post-mortem analysis via dmesg — a tool that reads the event log of the operating system kernel itself — confirmed the OOM event with the exact process identifier and the amount of memory consumed.

Recovery was fast only because of one component that had been running autonomously and independently of everything else: the rsync backup from 03:01 that same night was complete. Without loss. The backup was the only thing in the entire infrastructure architecturally isolated from the component whose failure it was supposed to survive.

Immediately after the reset: a swap file of 50% of total RAM and a Docker memory limit on the Ollama container — a hard ceiling of 33% RAM plus 12% swap. Both survive a reboot.

The insight I didn't have before that night: designing for recovery means explicitly modelling what happens when the assumption of normal operation doesn't hold. The difference isn't in complexity — in this case it was one command for swap and one Docker parameter. The difference is in attitude.

Katalog: When the Extension Lies

There is also a quieter kind of failure — no server crash, no OOM Killer. In the Katalog project, the system tried to read a set of .docx files using standard methods: unpacking the ZIP archive and converting through pandoc. Both tools failed with an error. The files carrying the .docx extension were in fact plain UTF-8 text Markdown files — no ZIP wrapper, no XML inside.

The failure wasn't visible to the user. It was hidden in an error from a tool called in the background. Without explicit exception catching and a fallback strategy — opening the file as plain text when unpacking fails — the system would have silently skipped the file or returned an empty result.

A file extension is not a contract. It describes intent, it does not guarantee content. Every interface to external data needs a fallback strategy — explicit, visible, and of equal value to the primary path.

The same project revealed another paradox, this time in my own documentation. For years I documented my projects in .docx format — the de facto standard for distributing documents, a format that every office and every client can open. A habit acquired through professional socialisation, worn in to the point of invisibility.

.docx is, in the world of documentation formats, one of the most opaque black boxes. A ZIP archive containing XML files, namespace declarations, relationship files — all of that just to store text with a heading and italics. Markdown is the opposite: a plain text file with a handful of syntax characters. It reads in any editor, it's searchable, it's versionable, it survives everything.

The solution wasn't either/or. I now document in Markdown format — and when I need to distribute, I generate a .docx. The same content, the right format for the right context.

And that shift unexpectedly brought me closer to the X-Ray attitude: the source of truth is now readable without a special tool. From within.